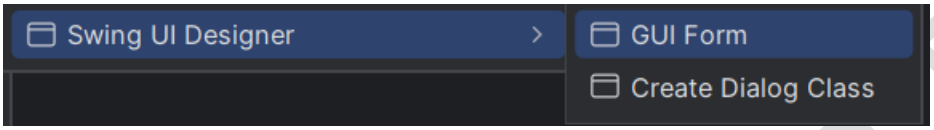
Hướng dẫn validate form trong Swing với IntelliJ
Giáo viên: Nguyễn Hùng Cường

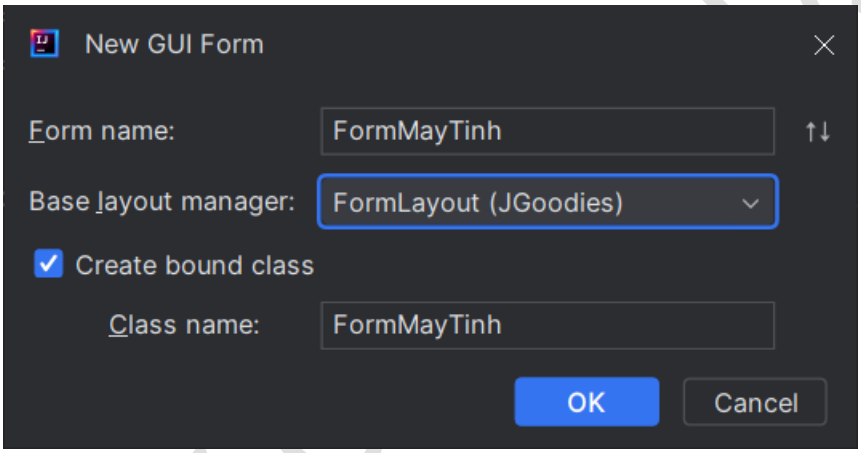
Bước 1: Tạo form giao diện

Mở một trình soạn thảo, rồi tạo một Java Project. Đặt tên và chọn đường dẫn chứa project vừa tạo.

Sau đó tạo mới một GUI Form như hình sau.

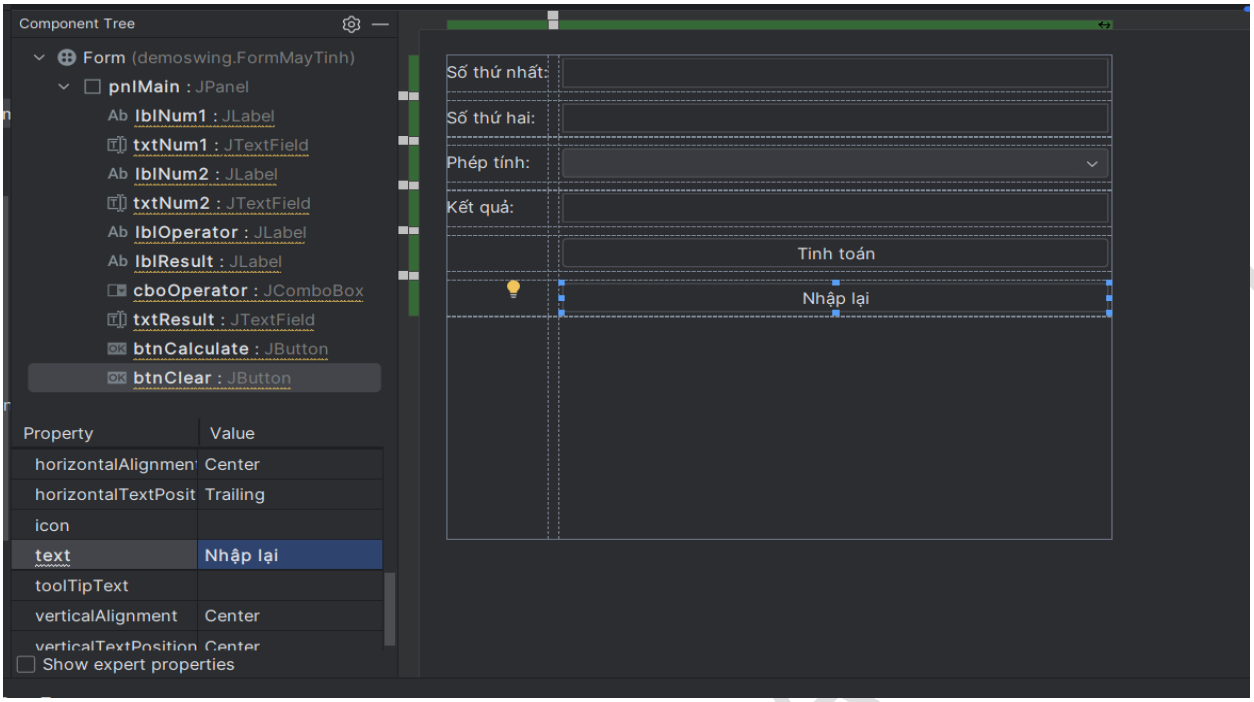


Ta đặt tên form, và chọn layout cho form. Ở đây ta chọn layout là FormLayout (có thể chọn layout khác).

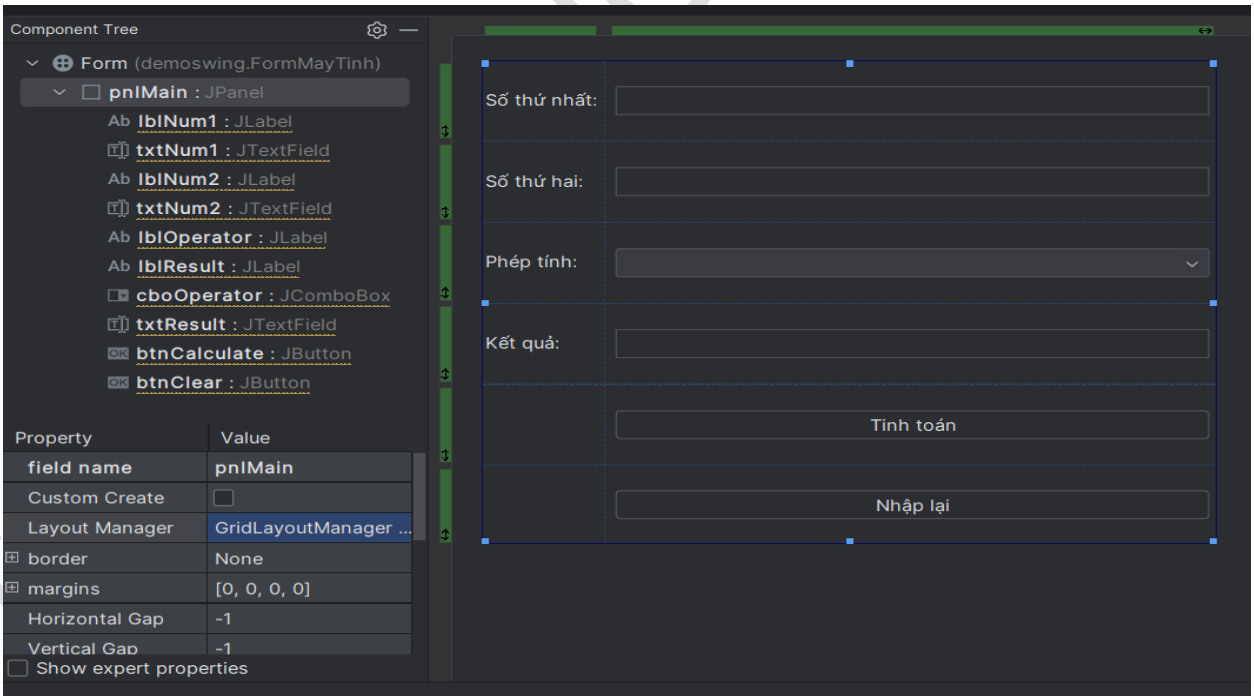


Sau đó, ta kéo thả các Swing components từ bên Palette panel sang form như hình bên dưới.

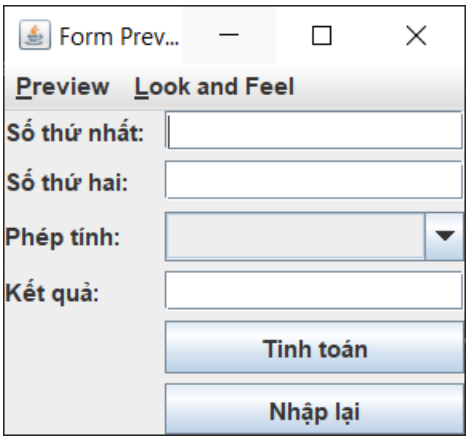
THẦY CƯỜNG LẬP TRÌNH



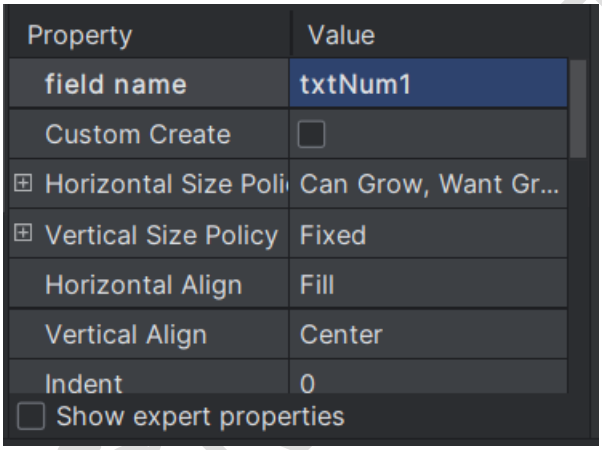
Tiếp theo, ta chọn một layout khác, ở đây ta chọn layout là GridLayout để hiển thị giao diện dưới dạng lưới, mỗi component sẽ được add vào một cell trong lưới. Ta thấy giao diện đã thay đổi như hình bên dưới.



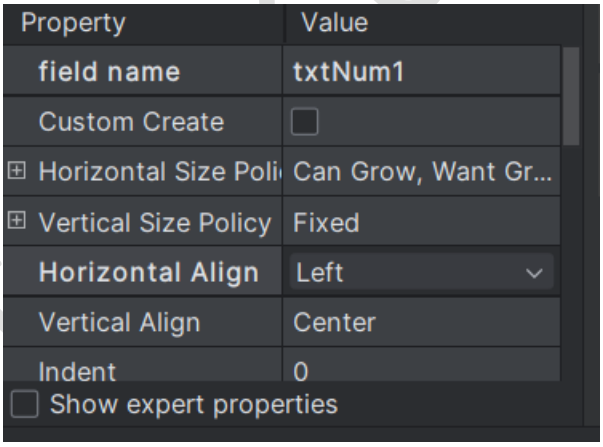
Ta có thể click chọn mục Preview để xem qua trước giao diện form như sau. Giao diện preview có thể khác so với giao diện thực của form, vì chưa thiết lập kích thước cho form.



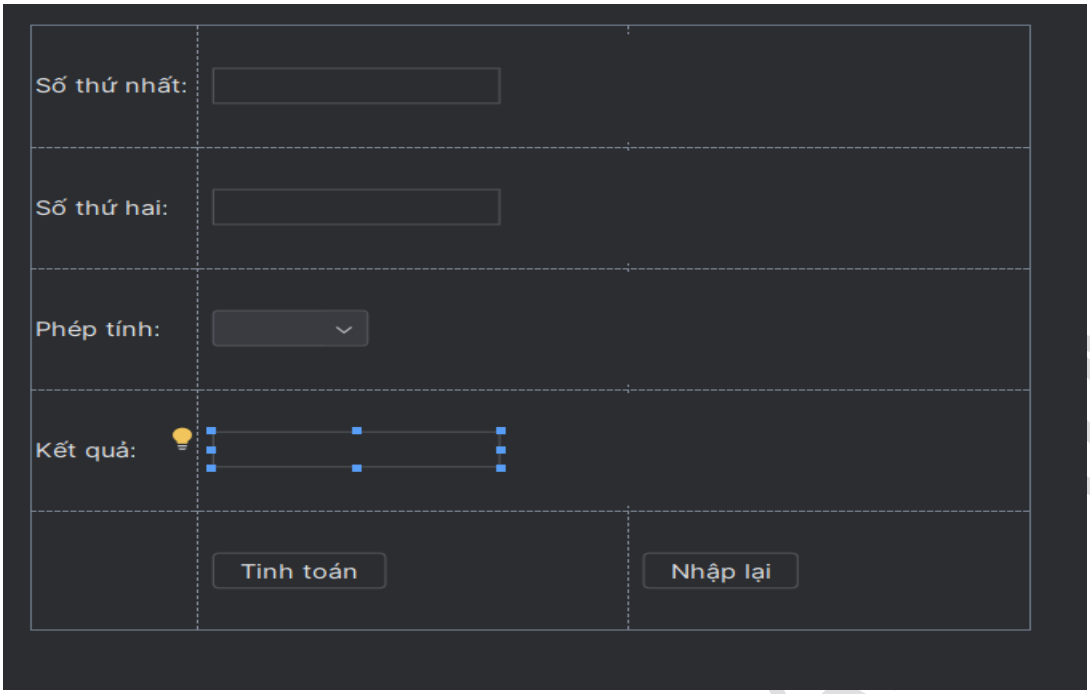
Mặc định, ta có thể thấy các component đã fill hết toàn bộ cell của form.



Nếu muốn cho các component nhỏ lại, ta thiết lập thuộc tính Horizontal Align từ Fill thành Left để căn trái cho component như sau.



Sau khi đã cập nhật lại giao diện, ta có thể thấy form đã thu gọn lại như hình bên dưới.



Bước 2: Viết mã xử lý cho form giao diện

Ta mở file .java ra, và sẽ thấy mã nguồn mặc định của file được tạo ra như sau.

```
1   package demoswing;
2
3   import javax.swing.*;
4
    2 usages
5   public class FormMayTinh {
        1 usage
6       private JPanel pnlMain;
        1 usage
7       private JLabel lblNum1;
        1 usage
8       private JTextField txtNum1;
        1 usage
9       private JLabel lblNum2;
        1 usage
10      private JTextField txtNum2;
        1 usage
11      private JLabel lblOperator;
        1 usage
12      private JLabel lblResult;
        1 usage
13      private JComboBox cboOperator;
        1 usage
14      private JTextField txtResult;
        1 usage
15      private JButton btnCalculate;
```

Ta cập nhật lại giao diện, cho class kế thừa từ lớp JFrame (là lớp thể hiện form giao diện), đồng thời viết mã để khởi tạo kích thước cho form như sau.

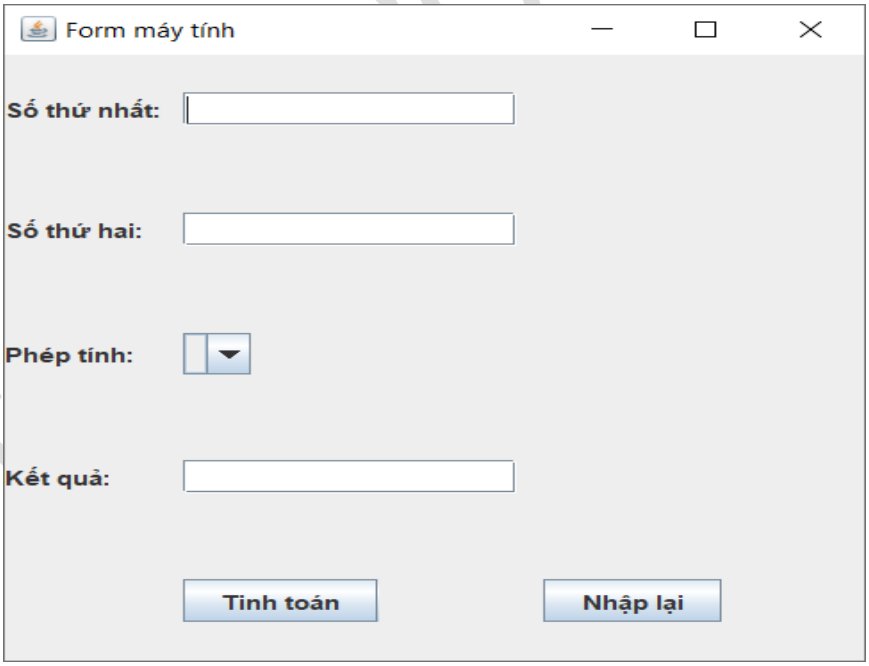
```
FormMayTinh.java
1   package demoswing;
2
3   import javax.swing.*;
4
5   public class FormMayTinh extends JFrame {
        2 usages
6       private JPanel pnlMain;
        1 usage
7       private JLabel lblNum1;
        1 usage
8       private JTextField txtNum1;
        1 usage
9       private JLabel lblNum2;
        1 usage
10      private JTextField txtNum2;
        1 usage
11      private JLabel lblOperator;
        1 usage
12      private JLabel lblResult;
        1 usage
13      private JComboBox cboOperator;
        1 usage
14      private JTextField txtResult;
        1 usage
15      private JButton btnCalculate;
```

Ta cũng đồng thời định nghĩa main method để thực thi form như sau.

```java
    private JButton btnCalculate;
    1 usage
    private JButton btnClear;

    1 usage
    public FormMayTinh() {
        this.setContentPane(pnlMain);

        this.setTitle("Form máy tính");
        this.setSize( width: 400,  height: 400);
        this.setVisible(true);
    }

    public static void main(String[] args) {
        FormMayTinh frmCalculator = new FormMayTinh();
    }
}
```

Bước 3: Thực thi chương trình và xem kết quả

Sau khi đã viết mã xong, hãy thực thi chương trình và xem kết quả. Ta sẽ thấy giao diện form đã được hiển thị như hình bên dưới.



Ta thấy combobox bị trống vì chưa có phần tử nào bên trong. Ta mở file .java, viết một method cho phép thêm một số phần tử vào combobox như sau.

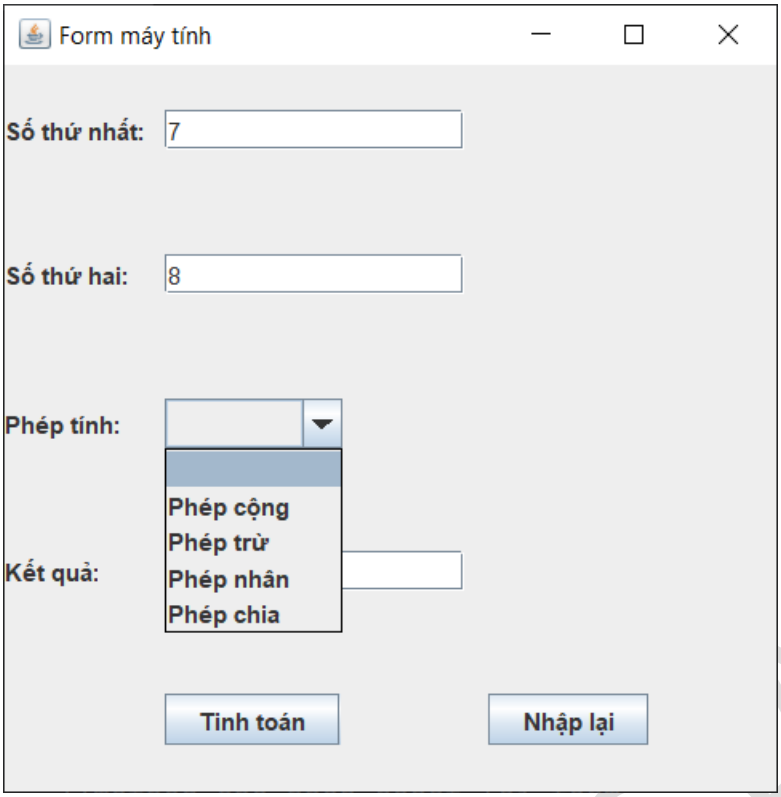
```
//method cho phep fill du lieu vào combobox
1 usage
private void fillFields() {
    cboOperator.addItem("");
    cboOperator.addItem("Phép cộng");
    cboOperator.addItem("Phép trừ");
    cboOperator.addItem("Phép nhân");
    cboOperator.addItem("Phép chia");
}
```

Ta gọi method fillFields() trong constructor để fill dữ liệu lên form như sau.

```
1 usage
public FormMayTinh() {
    this.setContentPane(pnlMain);

    this.setTitle("Form máy tính");
    this.setSize( width: 400,  height: 400);
    this.setVisible(true);

    fillFields();
}
```

Sau đó, thực thi lại chương trình, ta sẽ thấy combobox đã được khởi tạo như hình bên dưới đúng như mong muốn.

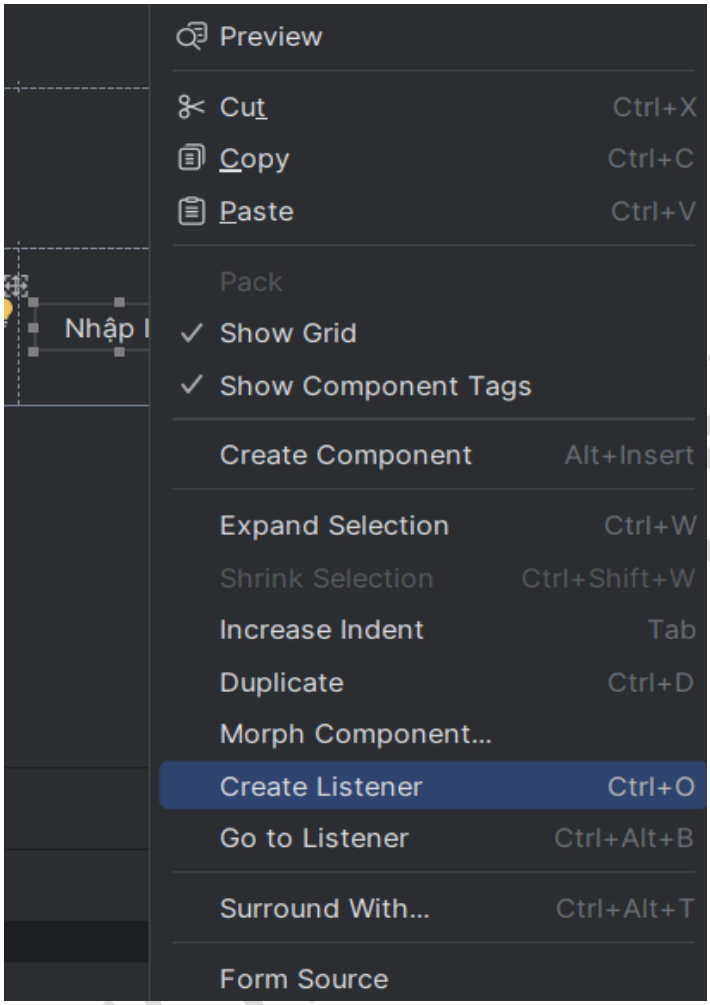
THẦY CÔ ÂP TRÌNH



Bước 4: Viết mã xử lý event cho form

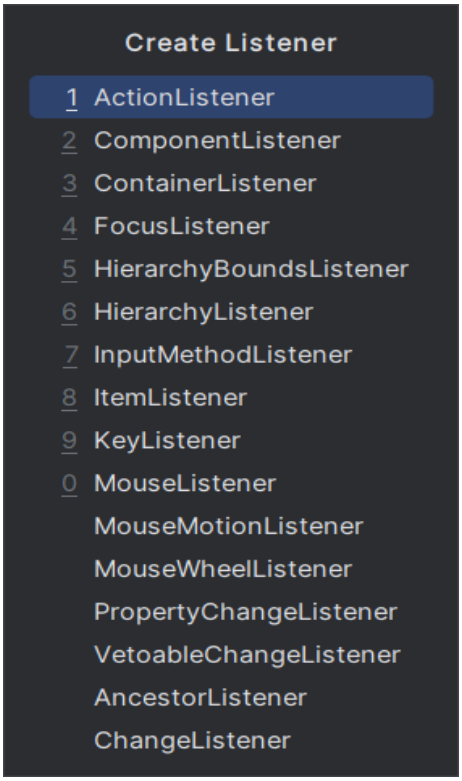
Để có thể xử lý event cho một phần tử trên form (hoặc ngay cả với form), ta click phải chuột lên component đó, rồi chọn Create Listener như sau. Có thể viết mã trực tiếp, hoặc sử dụng tổ hợp phím tắt Ctrl + O.

THẦY CƯỜNG LẬP TRÌNH

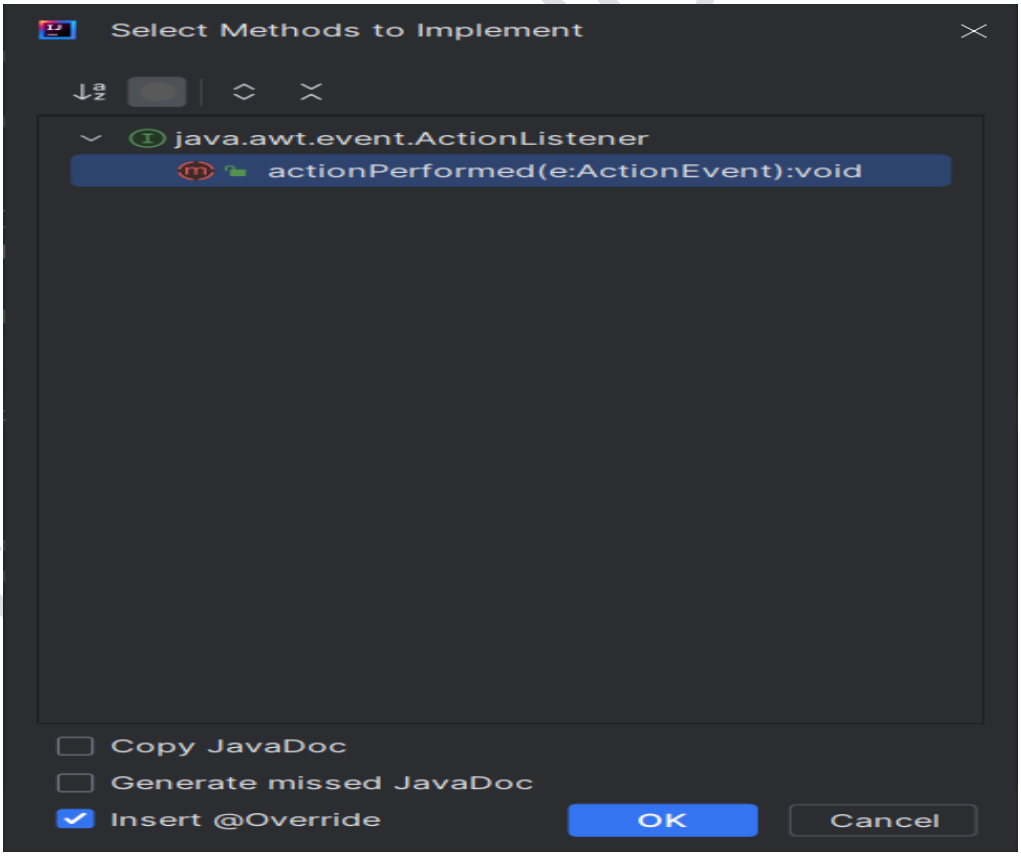


Trong hộp thoại mở ra, ta chọn loại listener tương ứng với sự kiện cần xử lý như hình sau. Ở đây ta chọn ActionListener để xử lý sự kiện ActionEvent, là sự kiện xảy ra mỗi khi ta click lên button hay menu.

THẦY CƯỜNG TRÌNH



Tiếp theo, ta chọn method để xử lý sự kiện được tạo ra.



Dưới đây là mã nguồn sau khi ta đã create listener ở bước trên.

```
//dang ky doi tuong ActionListener de xu ly ActionEvent
//moi khi nguoi dung click len button
btnClear.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {

    }
});
```

Ta định nghĩa method cho phép clear form, rồi gọi method này mỗi khi click lên button như sau.

```
    //dang ky doi tuong ActionListener de xu ly ActionEvent
    //moi khi nguoi dung click len button
    btnClear.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            clearFields();
        }
    });
}

//method cho phep reset lai form
1 usage
private void clearFields() {
    txtNum1.setText("");
    txtNum2.setText("");
    txtResult.setText("");
    //reset lai combobox
    cboOperator.setSelectedIndex(0);
}
```

Bước 5: Viết mã validate form và xử lý

Tiếp theo, ta định nghĩa method cho phép validate form, và xử lý tính toán giá trị của các phép tính mỗi khi người dùng click lên button như sau.

```
//method thuc hien viec tinh toan gia tri va validate form
1 usage
private void calculate() {
    String value1 = txtNum1.getText();
    if(value1.equals("")) {
        JOptionPane.showMessageDialog( parentComponent: this,  message: "Bạn cần nhập số thứ nhất !");
        txtNum1.requestFocus();
        return;
    }

    String value2 = txtNum2.getText();
    if(value2.equals("")) {
        JOptionPane.showMessageDialog( parentComponent: this,  message: "Bạn cần nhập số thứ hai !");
        txtNum2.requestFocus();
        return;
    }

    int selectedIndex = cboOperator.getSelectedIndex();
    if(selectedIndex == 0) {
        JOptionPane.showMessageDialog( parentComponent: this,  message: "Bạn cần chọn phép tính !");
        cboOperator.requestFocus();
        return;
    }
```

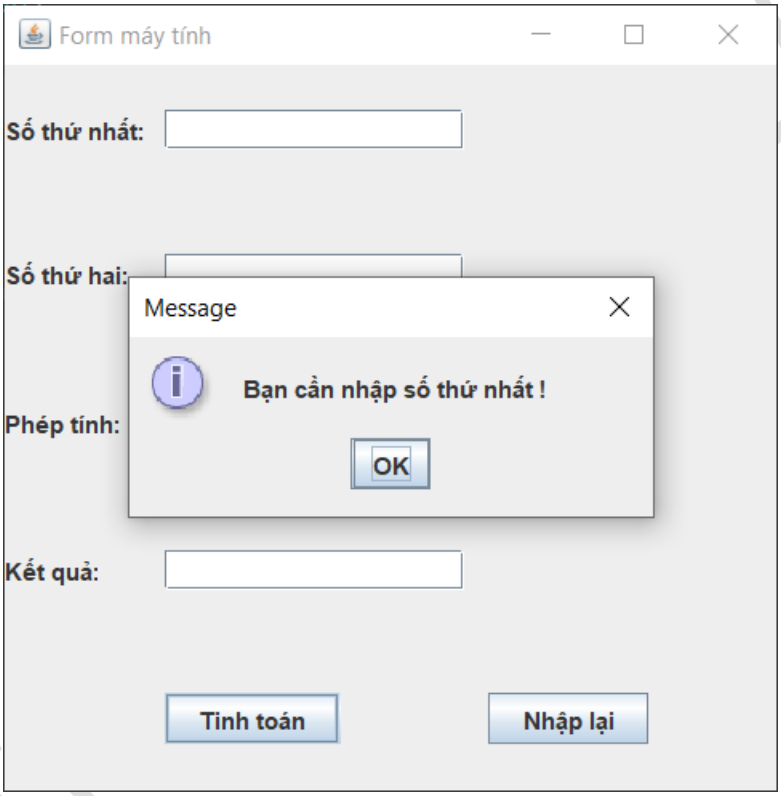
```
    int num1 = Integer.parseInt(value1);
    int num2 = Integer.parseInt(value2);

    //tinh toan dua tren phep tinh
    float result = 0;
    if(selectedIndex == 1) {
        result = num1 + num2;
    } else if (selectedIndex == 2) {
        result = num1 - num2;
    } else if (selectedIndex == 3) {
        result = num1 * num2;
    } else if (selectedIndex == 4) {
        result = (float) num1 / num2;
    }
    //hien thi ket qua
    txtResult.setText(String.valueOf(result));
}
```

Ta gọi method calculate() mỗi khi người dùng click lên button như sau.

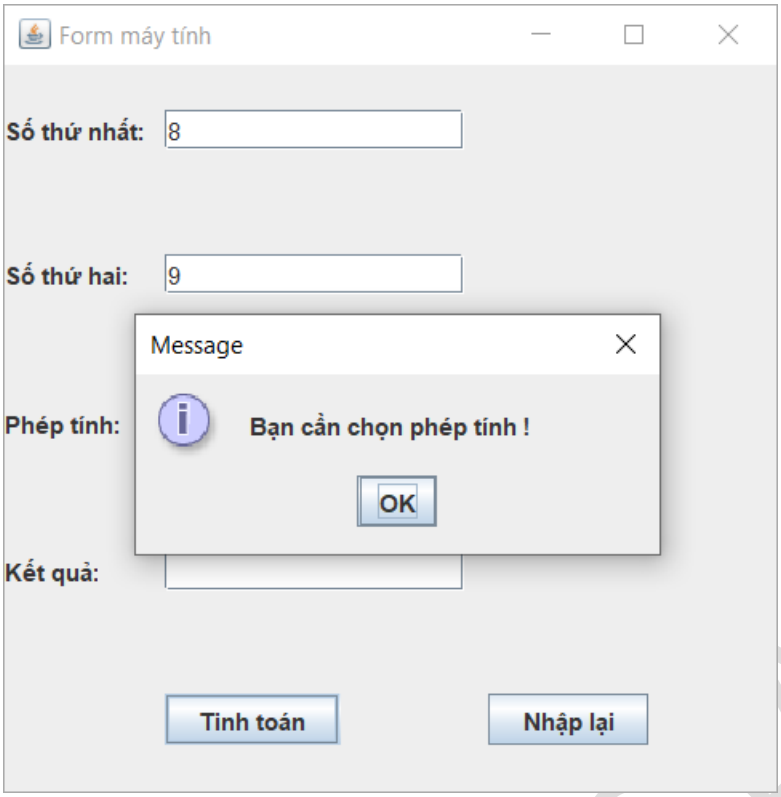
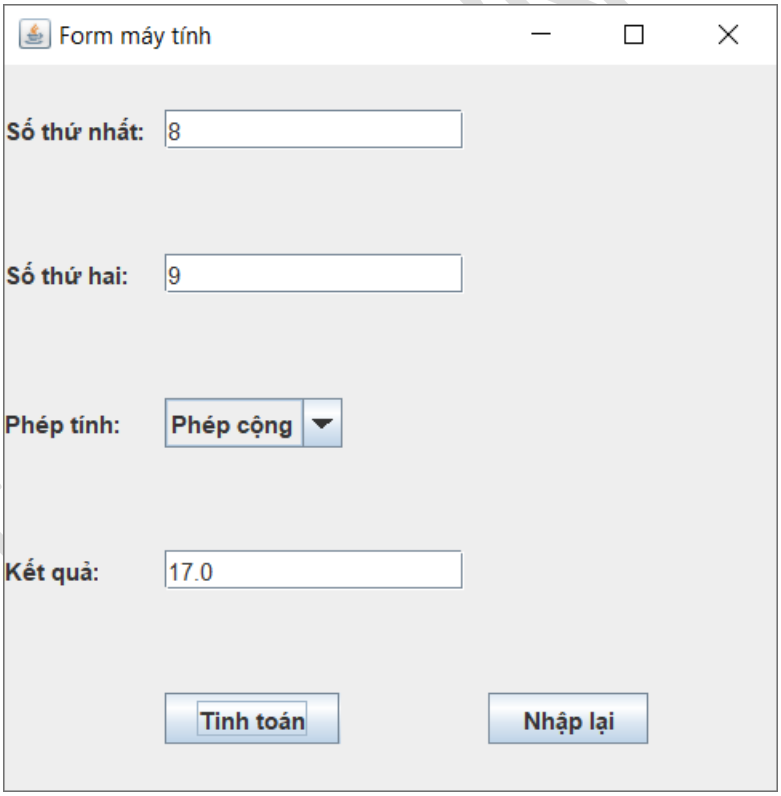
```
//dang ky listener de xu ly moi khi click len button Calculate
btnCalculate.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        calculate();
    }
});
```

Bước 6: Thực thi chương trình và xem kết quả

Sau khi đã viết mã xong, hãy thực thi chương trình và xem kết quả. Ta có thể thấy, mỗi khi người dùng click lên button Calculate mà không nhập dữ liệu, chương trình sẽ hiển thị thông báo lỗi như hình bên dưới đúng như mong muốn.



Nếu người dùng không chọn phép tính, chương trình cũng sẽ hiển thị thông báo lỗi như sau.



Sau khi người dùng đã nhập đầy đủ dữ liệu, ta thấy chương trình đã thực hiện tính toán và hiển thị kết quả như hình bên dưới đúng như mong muốn.